Predicting Song Popularity

Haoran Li
University of Michigan
Ann Arbor, MI
haoransl@umich.edu

***Abstract*—This project aims to predict the popularity of songs using metadata and audio features collected from Spotify. I explored a range of machine learning models and evaluated their performance using mean absolute error (MAE), root mean squared error (RMSE), and Test R Squared. Among these, CatBoost outperformed the rest due to its ability to handle high cardinality categorical features effectively. Feature importance from CatBoost revealed that artist, song age, and playlist genre were the most predictive features, while audio features like instrumentalness, tempo, and acousticness also contributed to song popularity. These results provide insights into listener preferences and considerations for marketing and recommendation strategies.**

I. Introduction

A. Background and Motivation

Millions of new songs are released every year, however, only a few actually become hits. The motivation of this project is that by accurately predicting the popularity of songs, platforms like Spotify can make better decisions on marketing and recommendation strategies. Additionally, identifying features or interactions of features that most strongly influence a song's popularity, such as its artist, age, energy, and acousticness, can help companies and artists make music that better aligns with listeners' preferences.

B. Project Goal

The goal of this project is to experiment with different models and preprocessing techniques to find the pipeline that mostly accurately predicts song popularity, evaluated using MAE, RMSE, and R^2.

C. Literature Review

In "Beyond Beats: A Recipe to Song Popularity?" Jung and Mayer investigated the use of machine learning to identify audio and metadata features that best predict a song's popularity score. They evaluated models like Ordinary Least Squares regression, Multivariate Adaptive Regression Splines (MARS), Random Forest, and XGBoost using MAE. Their results showed that Random Forest performed the strongest and "genre" was one of the most predictive features [2].

Pham, Kyauk, and Park addressed a similar problem in "Predicting Song Popularity," mainly through a binary classification lens. They focused extensively on feature engineering, such as computing the mean and variance of acoustic features across song segments, applying polynomial transformations for non-linear relationships, and using a bag-of-words model for text data (`artist_name`, `artist_id`, and `genre`). Since these techniques resulted in hundreds of features, they applied forward selection, backward selection, and regularization to reduce overfitting. For classification, they explored models like Logistic Regression, Linear Discriminant Analysis (LDA), Quadratic Discriminant Analysis (QDA), Support Vector Machines (SVM), and Multilayer Perceptron (MLP). They found that SVM with an RBF kernel performed the best in terms of F1 and AUC. For regression, they experimented with Linear Regression and Lasso Regression, focusing on tuning the regularization hyperparameter to improve MSE [3].

A related study by Saragih, "Predicting Song Popularity based on Spotify's Audio Features: Insights from the Indonesian Streaming Users," focused specifically on Indonesian streaming users. The preprocessing pipeline included dummy encoding for categorical variables, StandardScaler for normalization, cross-validation to prevent underfitting and overfitting, and grid search for hyperparameter tuning. They evaluated a wide range of models, including several tree-based models, MLP, K-Nearest Neighbors, SVM, and regression models for both classification and regression tasks. The results showed that Extra Trees Regressor and Random Forest Classifier outperformed the other models [4].

II. Method

A. Problem Formulation

Song popularity prediction was formulated as a supervised regression problem. The dataset used contains approximately 30,000 songs and 23 features collected from the Spotify Web API [1]. The input features include song metadata (e.g., `track_ID`, `track_artist`, `genre`, `release_year`) and audio characteristics (e.g., `acousticness`, `energy`, `key`). The output is the song popularity score. Model performance is evaluated using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R^2.

B. Pre-processing

After examining missing values, only 5 instances out of 30,000 had null values for features like `track_name` and `track_artist`. Since these are key features and cannot be imputed reliably, and they were only a negligible portion of the dataset, these rows were removed.

Duplicate handling was performed in two steps. First, any rows with duplicated `track_id` were removed. Then, for the remaining rows, duplicates based on the combination of `track_artist` and `track_name` were removed to ensure that only one instance per unique song remained.

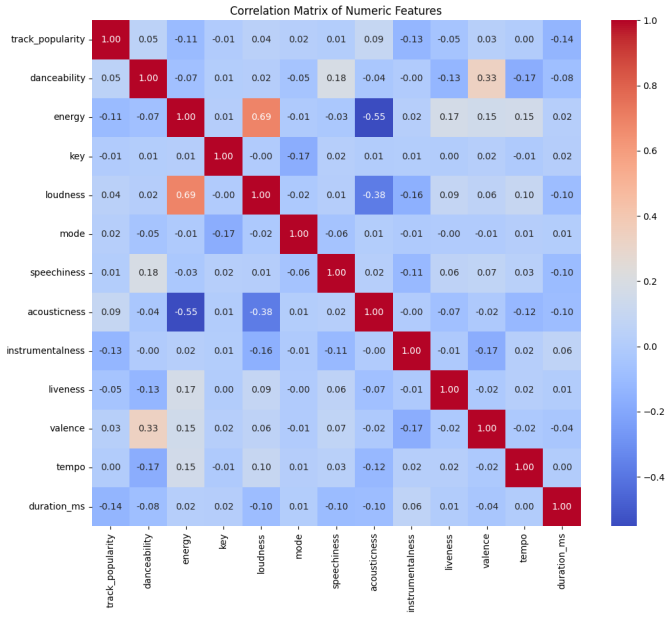

Fig. 1. Correlation Matrix for numerical features in the dataset

For feature selection, meaningless features such as `track_id`, `track_album_id`, and `playlist_id` were dropped. Looking at the correlation between features in Figure 1 revealed that `energy` and `loudness` had a relatively high correlation (0.69), so `loudness` was removed to reduce multicollinearity.

Categorical variables such as `key`, `playlist_genre`, and `mode` were encoded using one-hot encoding. After splitting the preprocessed dataset into training and testing sets, track artists were target-encoded for all models except CatBoost (described later). This was done because the dataset contains approximately 10,000 unique artists; one-hot encoding is impractical while target encoding can provide better estimates for artist influence. Since the features are on different scales, Min-Max normalization was applied to non-tree models to prevent features from dominating. Meanwhile, tree-based models can naturally handle unscaled features; therefore, no normalization was needed.

A new feature `age_of_song_in_years` was created using `track_album_release_date` to account for potential temporal patterns affecting popularity.

Finally, to investigate whether a song's name impacts popularity, a TF-IDF vectorization of `track_name` was created. This was only incorporated into non-tree models since TF-IDF creates a large and sparse vector of 5,000 tokens that can negatively impact tree-based model performances.

C. Models

For this project, I experimented with a range of traditional machine learning models, tree-based models, and deep learning models to compare their performances in predicting song popularity. Their hyperparameters were tuned manually.

The baseline model was Linear Regression, which fits the best linear relationship between the input features and the popularity score.

Lasso Regression builds on Linear Regression by adding L1 regularization parameter to penalize model complexity. This helps with implicit feature selection of useless features and reduces overfitting, which can potentially improve performance

K-Nearest Neighbors (KNN) predicts popularity by averaging values of its closest data points, which allows it to capture local and nonlinear patterns that linear models cannot find and lead to high performance.

Support Vector Regression (SVR) with an RBF kernel maps features into a high dimensional space to learn nonlinear relationships, allowing for more complex patterns to be fitted.

Multilayer Perceptron (MLP) uses neural networks to capture more complex feature interactions and nonlinear relationship between features and popularity, which could allow for better performance. However, it is very prone to overfitting.

Decision Trees split the data based on variance reduction and can perform implicit feature selection. Overfitting can be avoided by adjusting hyperparameters like maximum depth.

Random Forest improves performance over a single decision tree by averaging predictions from multiple trees. This reduces variances and is expected to perform better than a single tree.

Gradient Boosting builds multiple trees sequentially so that each tree corrects the errors from the previous tree. This decreases the bias from a single tree and is more likely to have a stronger performance.

LighGBM is a special type of Gradient Boosting that uses histogram binning and grows one leaf at a time rather than level-wise, making it more efficient for large datasets.

CatBoost is another special type of Gradient Boosting that is specifically designed for categorical variables with high cardinality. It uses ordered boosting and encodes categorical variables with only information from previous rows to achieve strong performance and reduce target leakage. Because of this, it doesn't need preprocessing of categorical variables beforehand.

III. Results

TABLE I
Model Performance Comparison

Model	MAE	RMSE	R^2_{train}	R^2_{test}
Linear Regression	15.778	20.897	0.621	0.178
Linear Regression (+ TF–IDF track_name)	16.750	22.274	0.716	0.066
Lasso Regression	15.779	20.898	0.621	0.178
Lasso Regression (+ TF–IDF track_name)	15.723	20.827	0.625	0.183
KNN	16.568	20.729	0.452	0.191
KNN (+ TF–IDF track_name)	17.754	21.687	0.276	0.114
SVR + RBF kernel	16.516	20.592	0.425	0.202
SVR + RBF kernel (+ TF–IDF track_name)	16.889	20.915	0.352	0.176
MLP	15.578	20.523	0.650	0.207
MLP (+ TF–IDF track_name)	16.163	21.057	0.689	0.165
Decision Tree	15.753	20.921	0.642	0.176
Random Forest	15.454	20.530	0.866	0.206
Gradient Boosting	15.420	20.454	0.805	0.212
CatBoost	15.056	19.005	0.579	0.320

From Table 1, it is evident that CatBoost outperforms all other models in terms of MAE, RMSE, and test R^2. This aligns with expectations since CatBoost is great at handling high cardinality categorical features, particularly `track_artist`, which is one of the most predictive features in this dataset. This demonstrates that CatBoost's native categorical variable encoding is stronger than target encoding and one-hot encoding approaches. After hyperparameter tuning using GridSearch and cross-validation, the best hyperparameters were found to be 8 for depth, 600 iterations, 5 for L2 leaf regularization, and a learning rate of 0.05.

Including TF-IDF vectorization of `track_name` worsened model performance for most models, except for Lasso Regression. This indicates that the 5,000 TF-IDF features introduced significant noise, suggesting that song name is not predictive of popularity in this dataset.

Gradient boosting models (Random Forest, Gradient Boosting, LightGBM, CatBoost) led to better performance compared to linear, KNN, kernel-based, and deep learning models. This is expected since gradient boosting sequentially corrects mistakes from the previous tree, decreasing bias and allowing for more accurate predictions.

Across all models, training R^2 values are substantially higher than testing R^2, which indicates that there is significant overfitting. Attempts to reduce overfitting through tuning regularization hyperparameters decreased performance, which suggests that the dataset is extremely noisy and challenging to predict.

When compared to the MAE reported in "Beyond Beats: A Recipe to Song Popularity? A machine learning approach," where the lowest MAE achieved was 16.31 using Random Forest, my model outperformed theirs with a lowest MAE of 15.056 [2]. In "Predicting Song Popularity," the reported MSE might be on a different scale and a different dataset might have been used, therefore these results are not directly comparable [3]. Similarly, in "Predicting song popularity based on Spotify's audio features: insights from the Indonesian streaming users," the best performing model achieved an R^2 of 0.6857 (unsure of test or train) and RMSE of 12.33, however, a different dataset was used resulting in it being not directly comparable [4].

Given CatBoost had the best performance, its feature importance was examined to identify the most predictive features. From Figure 2, `track_artist` appears as the most predictive feature, which is expected since an artist's popularity plays a huge role in a song's success. The age of the song is the second most important, indicating that temporal patterns do affect song popularity. The playlist genre is also significant, which highlights listeners' specific preferences. Among audio features, `instrumentalness`, `tempo`, `acousticness`, `duration`, and `energy` all contribute to predicting song popularity. While these are less predictive than artist and meta data, it still showcases listeners' preferences on various song attributes.

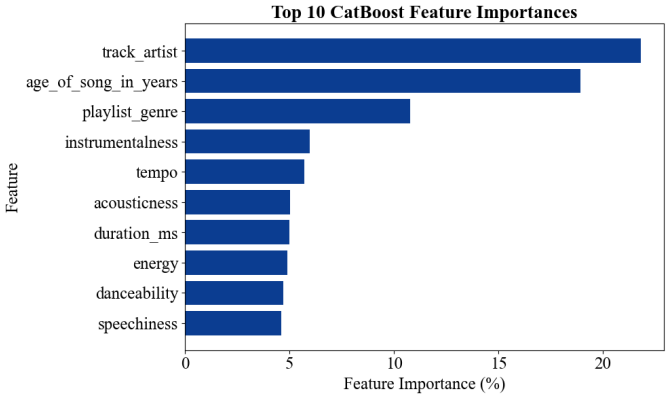

Fig. 2. Bar plot of top 10 CatBoost Feature Importances

IV. Conclusion

This project aimed to predict song popularity using song metadata and audio features. Among the models explored, CatBoost performed the best in terms of MAE, RMSE, and test R^2. These results suggest that when making decisions about marketing and recommendation strategies, companies should prioritize factors like the artist, song age, and genre as they have the strongest influence on song popularity. For artists making new music, attention to audio features like `instrumentalness`, `tempo`, and `acousticness` can help align songs with listeners' preferences. However, it is clear that datasets like this are inherently noisy to use and a song's popularity cannot be fully explained by metadata and audio features alone. External factors like marketing campaigns, streaming algorithms, cultural context, and lyrics likely play a significant role. For future studies, exploring additional data on these factors can further improve the performance of song popularity predictions.

References

[1] J. Arvidsson, "30,000 Spotify Songs," *Kaggle*, [Online]. Available: https://www.kaggle.com/datasets/joebeachcapital/30000-spotify-songs

[2] N. Jung and F. Mayer, "Beyond Beats: A Recipe to Song Popularity? A Machine Learning Approach," [Online]. Available: https://arxiv.org/pdf/2403.12079

[3] J. Pham, E. Kyauk, and E. Park, "Predicting Song Popularity," [Online]. Available: https://cs229.stanford.edu/proj2015/140_report.pdf

[4] H. S. Saragih, "Predicting song popularity based on Spotify's audio features: insights from the Indonesian streaming users," *Journal of Management Analytics*, vol. 10, no. 4, pp. 693–709, Jul. 2023, doi: https://doi.org/10.1080/23270012.2023.2239824

Disclosure: ChatGPT was used to help with coding and edit the final report.